

ЗАНЯТИЕ 4 (1:30)

Документы

ПРОДОЛЖИТЕЛЬНОСТЬ

Ориентировочная продолжительность занятия – 1 час 30 минут.

Что такое документ.....	122
Формы документа.....	126
Теория: типы данных, типобразующие объекты конфигурации	126
Документ «Приходная накладная»	127
Добавление документа	128
Реквизиты ссылочного типа	130
Свойство «Значение заполнения» реквизита объекта конфигурации	130
Проверка заполнения табличной части	132
Добавление приходных накладных	135
Теория: справочники и документы	138
Автоматический пересчет суммы в строках документа.....	139
Форма документа	139
Обработчик события.....	143
Одна процедура для обработки нескольких событий.....	147
Общий модуль	147
Документ «Оказание услуги»	150
Контрольные вопросы	154

На этом занятии мы познакомимся с объектом конфигурации *Документ*. Вы узнаете, для чего он нужен, какова его структура и какими основными свойствами он обладает.

Затем мы создадим несколько документов и покажем, каким образом разработчик может задавать собственные алгоритмы выполнения тех или иных действий, связанных с работой документа.

Кроме этого, вы узнаете, как создать форму документа, познакомитесь с некоторыми конструкциями встроенного языка и узнаете, что такое типобразующие объекты конфигурации.

Что такое документ

Объект конфигурации *Документ* предназначен для описания информации о совершенных хозяйственных операциях или о событиях, произошедших в жизни организации вообще. Как правило, в работе любой фирмы используются такие документы, как приходные накладные, приказы о приеме на работу, платежные поручения, счета и т. д. Свойства и структура этих документов описываются в объектах конфигурации *Документ*, на основе которых платформа создает в базе данных таблицы для хранения информации из этих документов.

Логика работы документов отличается от логики работы других объектов конфигурации. Документ обладает способностью *проведения*. Факт проведения документа означает, что событие, которое он отражает, повлияло на состояние учета.

До тех пор, пока документ не проведен, состояние учета неизменно, и документ не более чем черновик, заготовка. Как только документ будет проведен, изменения, вносимые документом в учет, вступают в силу и состояние учета будет изменено.

Поскольку документ вносит изменения в состояние учета, он всегда «привязан» к конкретному моменту времени. Это позволяет отражать в базе данных фактическую последовательность событий.

Следующим важным фактом, вытекающим из двух предыдущих, является то, что система «1С:Предприятие» имеет механизмы, позволяющие отслеживать правильность состояния учета. Предположим, что мы изменили один из проведенных ранее документов и снова провели его задним числом. В этом случае система «1С:Предприятие» способна отследить, повлияют ли внесенные нами изменения

на последующие проведенные документы, и, если это так, система способна перепровести необходимые документы.

В процессе работы пользователь может самостоятельно создавать новые документы – приходные и расходные накладные, счета и т. п.

В базе данных каждый документ представляет собой отдельную запись в основной таблице, хранящей информацию об этом виде документов (рис. 4.1).

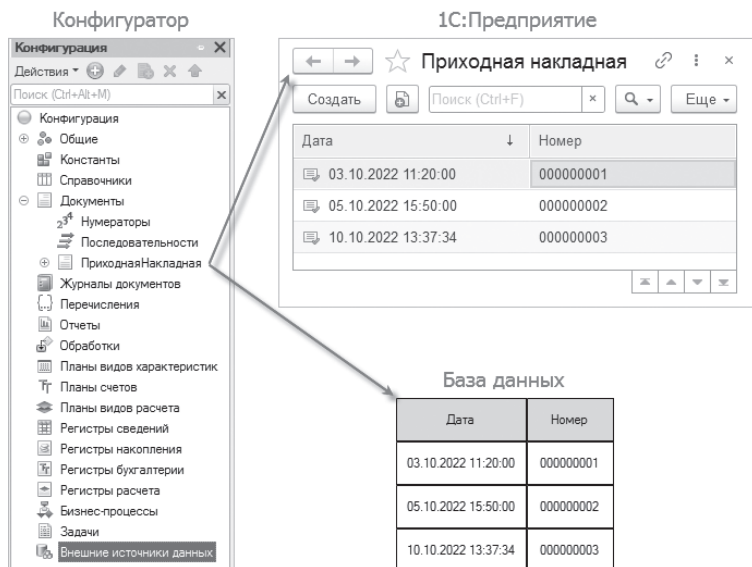


Рис. 4.1. Стандартные реквизиты документа «Приходная накладная» в режиме «Конфигуратор», в режиме «1С:Предприятие» и в базе данных

Каждый документ, как правило, содержит информацию, которая подробнее описывает этот документ. Например, каждый документ Приходная накладная может содержать информацию о поставщике товаров, складе, на который приходится товар, и т. д.

Набор такой информации является одинаковым для всех документов одного вида, и для описания такого набора используются реквизиты объекта конфигурации Документ, являющиеся подчиненными объектами конфигурации. Большинство реквизитов разработчик создает самостоятельно, однако у каждого объекта конфигурации Документ по умолчанию существуют стандартные реквизиты.

Два наиболее важных из них – это **Дата** и **Номер**. Поскольку тип данных **Дата** содержит дату и время с точностью до секунды, этот реквизит и определяет в основном положение документа на оси времени (рис. 4.2).

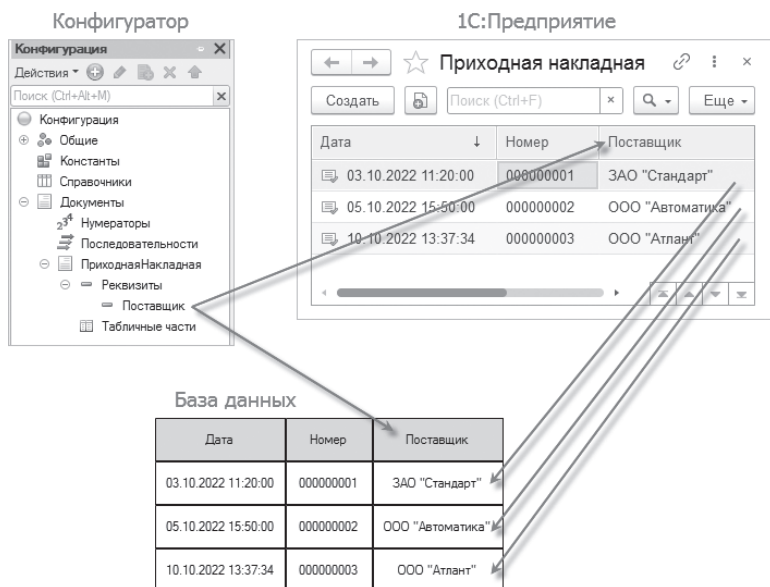


Рис. 4.2. Реквизит «Поставщик» документа «Приходная накладная» в режиме «Конфигуратор», в режиме «1С:Предприятие» и в базе данных

Кроме этого, каждый документ содержит, как правило, некоторый набор информации, которая одинакова по своей структуре, но различна по количеству и предназначена для разных документов. Так, например, каждый документ **Приходная накладная** может содержать список приходуемых товаров.

Для описания подобной информации служат табличные части объекта конфигурации **Документ**. В этом случае в базе данных будут созданы дополнительные таблицы для хранения табличных частей, подчиненных конкретному документу (рис. 4.3).

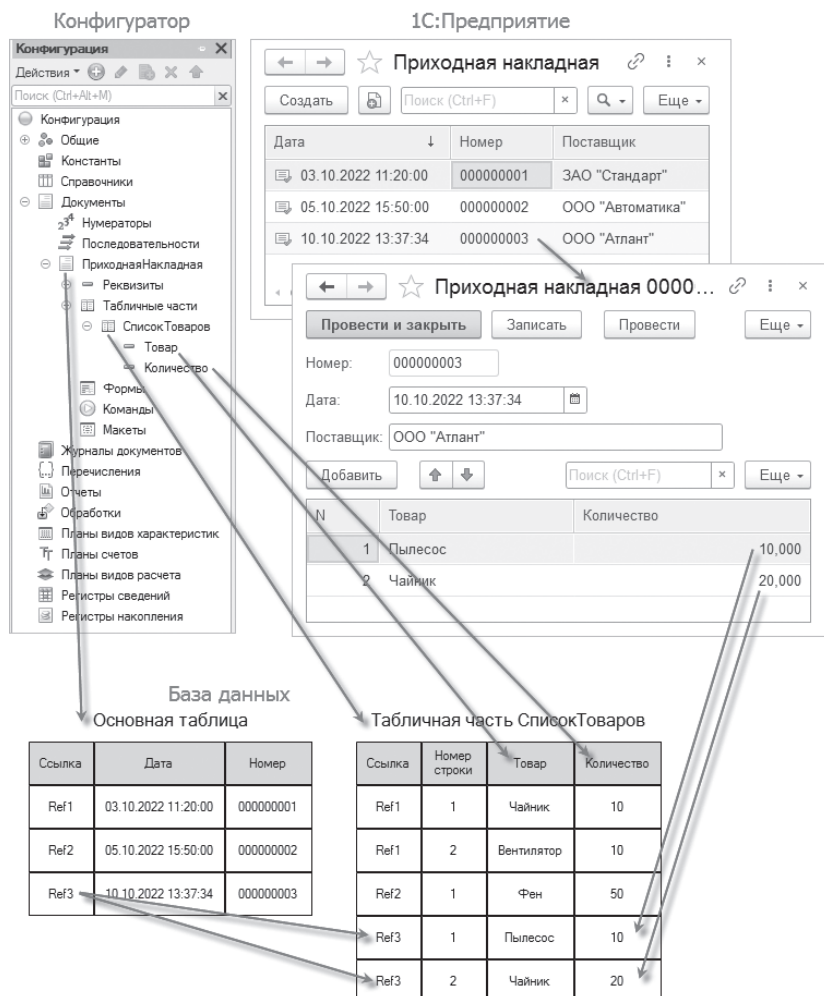


Рис. 4.3. Табличная часть «Список товаров» документа «Приходная накладная» в режиме «Конфигуратор», в режиме «1С:Предприятие» и в базе данных

Формы документа

Для визуализации документа существует несколько основных форм, которые, как мы уже говорили, имеют несколько вариантов названий (табл. 4.1).

Таблица 4.1. Основные формы документа

В контекстном меню и в палитре свойств	В конструкторе форм	На закладке формы
Форма объекта	Форма документа	Документа
Форма списка	Форма списка документа	Списка
Форма выбора	Форма выбора документа	Выбора

УЗНАЙ БОЛЬШЕ!

О структуре объектов встроенного языка, предназначенных для работы с документами, можно прочитать в разделе «Краткий справочник разработчика. Документы».

Теория: типы данных, типобразующие объекты конфигурации

Прежде чем мы приступим к практическому созданию документов, необходимо сделать отступление о том, какие типы данных могут использоваться в системе «1С:Предприятие».

На предыдущем занятии, когда мы создавали реквизиты справочников или табличных частей, мы всегда указывали тип значения, которое может принимать этот реквизит. Это были *примитивные* типы данных: Число, Строка, Дата и Булево. Примитивные типы данных изначально определены в системе, и их набор ограничен.

Наряду с такими изначально определенными в любой конфигурации типами могут существовать типы данных, определяемые только конкретной конфигурацией. То есть такие типы, которые не присутствуют в конфигурации постоянно, а появляются в результате того, что добавлены некоторые объекты конфигурации.

Например, после того как мы создали объект конфигурации Справочник Склады, сразу же появилось несколько новых типов данных, связанных с этим справочником. Среди них, например, СправочникСсылка.Склады. И если теперь мы укажем какому-

либо реквизиту этот тип данных, то сможем хранить в нем ссылку на конкретный объект справочника Склады.

Объекты конфигурации, которые могут образовывать новые типы данных, называются *типообразующими*.

Например, после создания нового справочника Номенклатура становятся доступны следующие типы данных:

- СправочникМенеджер.Номенклатура,
- СправочникСсылка.Номенклатура,
- СправочникОбъект.Номенклатура,
- СправочникВыборка.Номенклатура.

Следует еще раз отметить, что эти типы данных не поддерживаются платформой изначально и существуют только в конкретном прикладном решении.

Это небольшое отступление было необходимо потому, что уже при создании первого документа мы столкнемся с использованием типов данных СправочникСсылка.Склады и СправочникСсылка.Номенклатура, которые появились в нашей конфигурации в результате создания объектов конфигурации Справочник Склады и Номенклатура.

Документ «Приходная накладная»

После того как мы познакомились с объектом конфигурации Документ, создадим несколько таких объектов, чтобы иметь возможность фиксировать события, происходящие в нашем ООО «На все руки мастер».

Одними из самых популярных услуг нашего предприятия является ремонт телевизоров и установка стиральных машин. И в том, и в другом случае требуются некоторые материалы, которые расходуются в процессе оказания этих услуг. Поэтому двумя важнейшими событиями в хозяйственной жизни нашей организации будут являться поступление материалов и оказание услуг.

Для отражения этих событий в базе данных мы создадим два документа: Приходная накладная и Оказание услуги.

Документ Приходная накладная будет фиксировать факт поступления в нашу организацию необходимых материалов, а документ Оказание услуги – фиксировать оказание услуг и расход материалов, которые используются при оказании этих услуг.

В режиме «Конфигуратор»

Добавление документа

Откроем конфигуратор и добавим новый объект конфигурации Документ. На закладке Основные зададим имя документа – ПриходнаяНакладная. На основании имени платформа автоматически заполнит его синонимом.

Здесь же определим, как будет представлен документ в интерфейсе «1С:Предприятия». Представление объекта задавать не будем, вместо него будет использоваться Синоним объекта. Это нам вполне подходит, так как мы задали его имя в единственном числе.

Представление списка, наоборот, зададим во множественном числе как Приходные накладные (рис. 4.4).

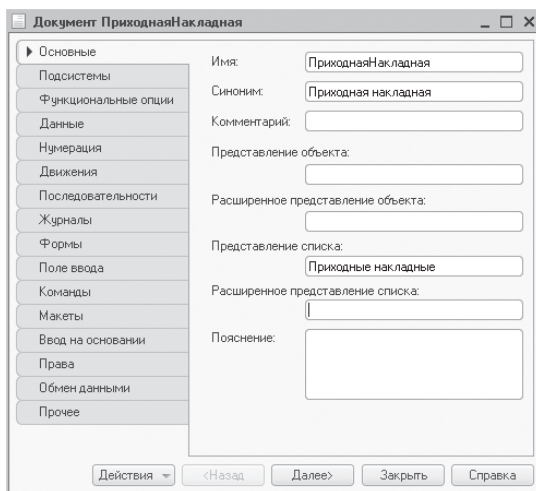


Рис. 4.4. Установка основных свойств документа

Нажмем Далее и перейдем на закладку Подсистемы.

По логике нашей конфигурации список приходных накладных должен быть доступен в разделах Учет материалов и Бухгалтерия. Действительно, к первому разделу этот документ имеет прямое отношение, а для бухгалтерского анализа всегда может понадобиться список документов, отражающих поступление материалов.

Поэтому отметим в списке подсистем эти подсистемы (рис. 4.5).

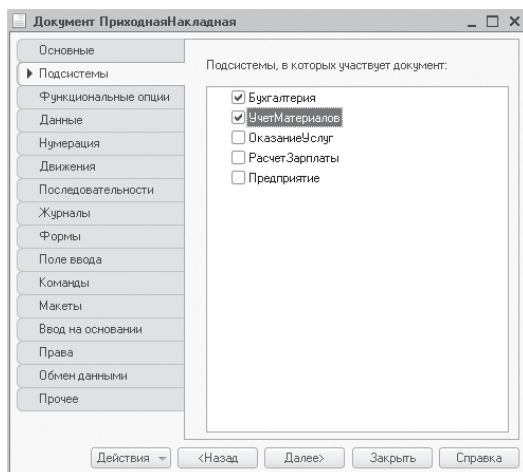


Рис. 4.5. Определение списка подсистем, в которых будет отражаться документ

Перейдем на закладку **Данные** и создадим реквизит документа с именем **Склад**. Для этого нажмем кнопку **Добавить** над списком реквизитов документа (рис. 4.6).

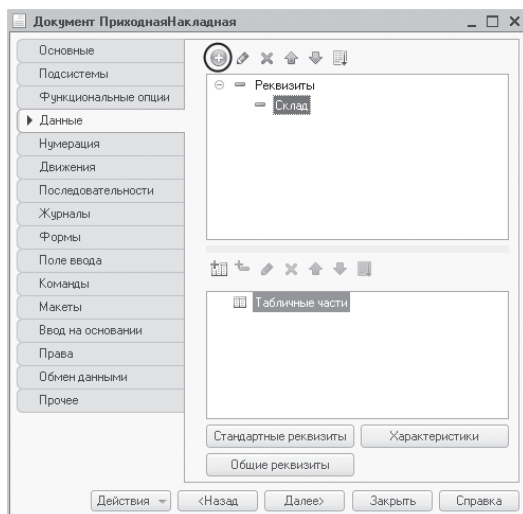


Рис. 4.6. Создание реквизита документа

Реквизиты ссылочного типа

Выберем для реквизита ссылочный тип данных СправочникСсылка.Склады. Этот тип стал доступен в конфигурации после создания объекта конфигурации Справочник Склады (рис. 4.7).

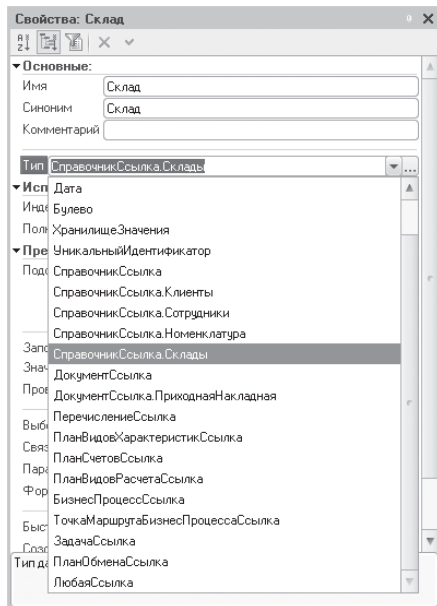


Рис. 4.7. Свойства реквизита документа

Свойство «Значение заполнения» реквизита объекта конфигурации

Теперь покажем, как можно облегчить жизнь пользователя при приходовании материалов. Работа в автоматизируемой нами фирме построена таким образом, что, как правило, все поступающие товары приходятся на основной склад.

Поэтому в палитре свойств для созданного нами реквизита Склад документа найдем свойство Значение заполнения.

В качестве значения этого свойства выберем predetermined element of the warehouse dictionary – Main.

Таким образом, при создании нового документа склад будет сразу заполняться значением Основной, и пользователю не придется делать это вручную (рис. 4.8).

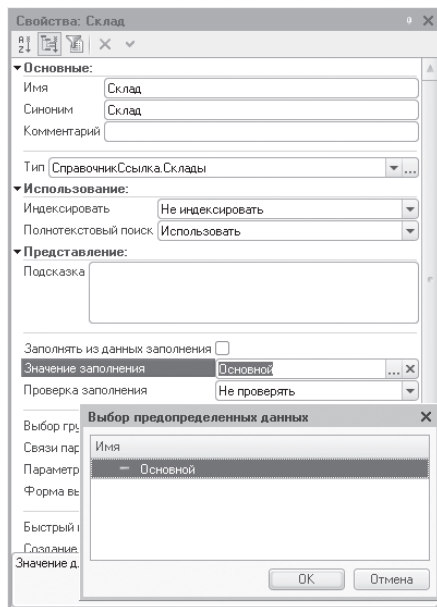


Рис. 4.8. Выбор значения заполнения по умолчанию для реквизита «Склад»

После этого добавим в документ табличную часть с именем Материалы. Для этого нажмем кнопку Добавить табличную часть над списком табличных частей документа (рис. 4.9).

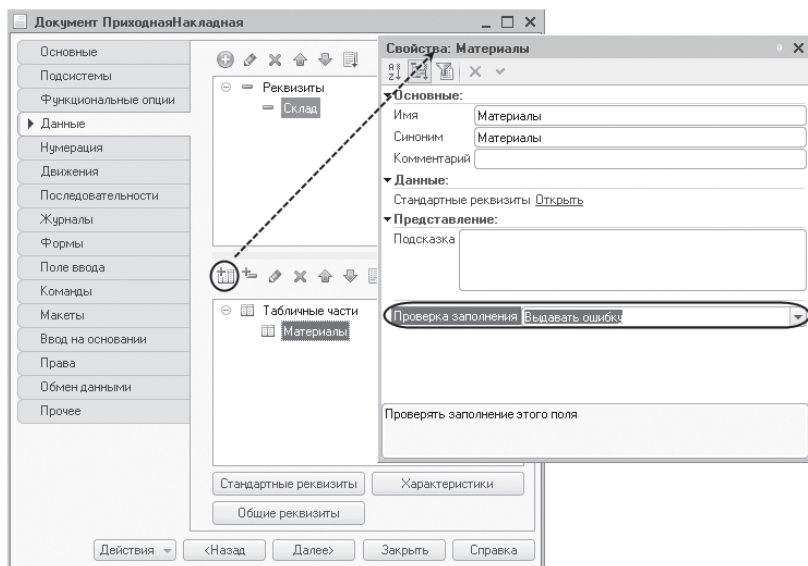


Рис. 4.9. Создание новой табличной части документа

Проверка заполнения табличной части

Кроме имени табличной части установим свойство Проверка заполнения в значение Выдавать ошибку. Тем самым мы задаем условие, что документ Приходная накладная обязательно должен содержать табличную часть, то есть список приходуемых материалов. Иначе будет выдано сообщение об ошибке, и документ не будет сохранен.

Создадим реквизиты табличной части Материалы. Для этого нажмем кнопку Добавить реквизит в разделе описания табличных частей документа (рис. 4.10):

- Материал, тип СправочникСсылка.Номенклатура;
- Количество, тип Число, длина 15, точность 3, неотрицательное;
- Цена, тип Число, длина 15, точность 2, неотрицательное;
- Сумма, тип Число, длина 15, точность 2, неотрицательное.

Для каждого реквизита табличной части также установим свойство Проверка заполнения в значение Выдавать ошибку. Тем самым при записи документа будет проверяться на заполнение не только табличная часть в целом, но и ее отдельные реквизиты.

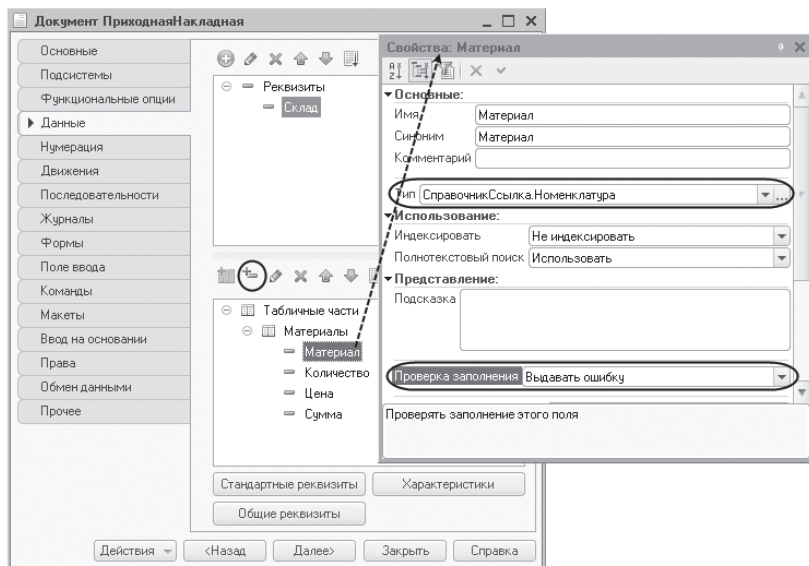


Рис. 4.10. Создание реквизитов табличной части документа

Перейдем на закладку Нумерация и убедимся, что свойство Авто-нумерация включено. Это обеспечит автоматическую генерацию уникальных номеров для создаваемых нами документов.

В заключение отредактируем командный интерфейс, чтобы в разделе Учет материалов была доступна команда создания новых документов.

Для этого откроем окно редактирования объекта конфигурации Подсистема УчетМатериалов и нажмем кнопку Командный интерфейс. В открывшемся окне Командный интерфейс отразятся все команды этой подсистемы.

В группе Панель действий.Создать включим видимость у команды Приходная накладная: создать (рис. 4.11).

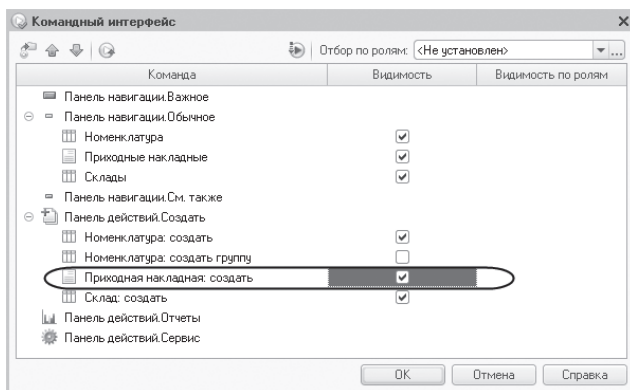


Рис. 4.11. Окно настройки командного интерфейса подсистемы

В режиме «1С:Предприятие»

Запустим «1С:Предприятие» в режиме отладки и протестируем получившийся документ.

В открывшемся окне «1С:Предприятия» мы видим, что в разделах Бухгалтерия и Учет материалов появилась команда Приходные накладные для открытия списка приходных накладных (рис. 4.12).

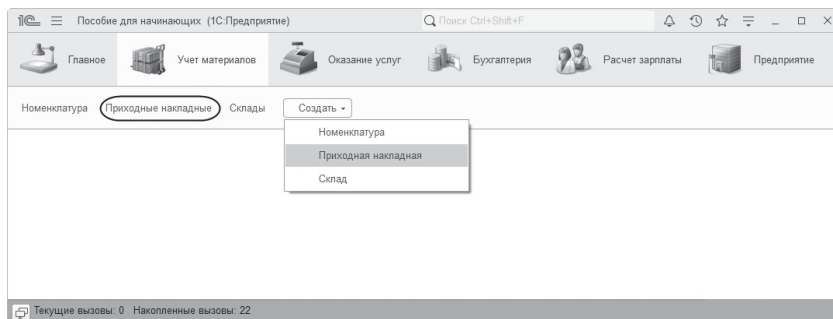


Рис. 4.12. Раздел «Учет материалов»

Название этой команды определяется свойством Представление списка, которое мы задали для этого документа.

Кроме того, в разделе Учет материалов в подменю Создать добавилась команда Приходная накладная для создания новых документов

этого вида. Название этой команды определяется синонимом, так как Представление объекта мы для этого документа не задавали (см. рис. 4.12).

Добавление приходных накладных

Пока в нашей базе данных нет ни одного документа Приходная накладная, поэтому выполним команду Приходная накладная из подменю Создать в разделе Учет материалов и создадим новую приходную накладную.

Откроется форма документа. Заголовок этой формы Приходная накладная совпадает с синонимом документа.

Система автоматически подставит текущую дату создания документа и нулевое время, так как документ еще не проведен. В качестве времени документа при оперативном проведении ему присваивается оперативная отметка времени.

Поле Номер не заполнено, но система сама сгенерирует для нового документа уникальный номер, так как свойство Автонумерация для документа включено по умолчанию. Новый номер будет сохранен в момент записи документа.

Обратите внимание, что склад уже заполнен значением Основной, как мы и задали в свойствах этого реквизита.

Нам осталось только заполнить табличную часть приходной накладной материалами для ремонта телевизоров так, как показано на рисунке (рис. 4.13).

N	Материал	Количество	Цена	Сумма
1	Строчный трансформатор GoldStar	10,000	270,00	2 700,00
2	Строчный трансформатор Samsung	10,000	600,00	6 000,00
3	Транзистор Philips 2N2369	10,000	3,00	30,00

Рис. 4.13. Создание нового документа «Приходная накладная № 1»

Обратите внимание, что при переходе к еще незаполненной колонке **Материал** (в табличной части документа) автоматически открывается окно выбора значения из справочника **Номенклатура**, так как этот реквизит имеет ссылочный тип данных и ссылается на справочник **Номенклатура** (см. рис. 4.13).

Как только мы начнем вводить название материала в это поле, платформа автоматически найдет материалы, наименование которых начинается с введенных нами символов, и предложит их нам для выбора. А также при нажатии кнопки **Показать все** открывается форма для выбора элементов этого справочника.

Нажмем **Провести и закрыть**.

Документ будет сохранен и проведен, ему будет присвоен автоматически сгенерированный системой номер и текущее время проведения документа.

Аналогичным образом мы создадим второй документ, который будет приходить на Основной склад материалы для установки стиральных машин (рис. 4.14).

N	Материал	Количество	Цена	Сумма
1	Кабель электрический	5,000	20,00	100,00
2	Шланг резиновый	5,000	100,00	500,00

Рис. 4.14. Создание нового документа «Приходная накладная № 2»

Нажмем **Провести и закрыть**.

Документ будет сохранен и проведен, ему будет присвоен автоматически сгенерированный системой номер и текущее время проведения документа.

Обратите внимание, что при вводе нового документа табличная часть в целом и каждая ее колонка подсвечены красным пунктиром. Это значит, что для них выполняется проверка заполнения. Если не ввести ни одной строки в табличную часть документа или оставить незаполненной какую-либо колонку табличной части и попытаться провести документ, то будет получено сообщение об ошибке (рис. 4.15).

Теория: справочники и документы

Интересно обратить внимание на разницу в употреблении единственного и множественного числа при именовании объектов вида Справочник и Документ.

Если вы откроете дерево типовой конфигурации, то увидите, что все объекты, расположенные в ветке Справочники, как правило, именованы во множественном числе. В ветке же Документы, как правило, в единственном числе.

Может сложиться впечатление, что, создавая объект конфигурации Справочник, мы делаем хранилище для элементов этого справочника, а создавая объект конфигурации Документ, – некий шаблон одного конкретного документа. На самом деле это не так.

Создавая в конфигураторе объект вида Справочник, мы даем ему наименование во множественном числе (Товары). При этом подразумевается, что в базе данных этот объект будет состоять из элементов, описывающих конкретные товары (в единственном числе).

Создавая в конфигураторе объект вида Документ, мы даем ему наименование в единственном числе (ПриходнаяНакладная), однако на самом деле мы создаем такое же хранилище, как и в случае со справочником. Каждая запись этого хранилища будет описывать один документ, одну приходную накладную (в единственном числе). Поэтому концептуально правильно было бы в конфигураторе задавать наименование объекта вида Документ во множественном числе, подчеркивая тем самым описание набора документов этого вида (например, ПриходныеНакладные).

Однако психология человека такова, что, открывая ветку Документы, он ожидает увидеть перечисление их в единственном числе, а никак не во множественном. Так происходит потому, что в реальной жизни трудно найти подходящий термин для описания совокупности документов одного вида (совокупность записей одного вида обозначить гораздо проще – справочник, план и т.д.). Поэтому соответствующая ветка объектов конфигурации имеет название Документы, а объекты конфигурации, создаваемые в этой ветке, именуются в единственном числе, хотя, по сути, сама ветка содержит описания хранилищ документов разных видов, а каждый элемент в этой ветке описывает набор всех документов одного вида.

Автоматический пересчет суммы в строках документа

Наверняка вы обратили внимание на то, что при заполнении документа приходится вводить сумму в каждой строке. Это неудобно, и возникает естественное желание автоматизировать работу документа так, чтобы сумма вычислялась автоматически каждый раз при изменении цены или количества материалов в строке.

Это совсем не сложно, и для этого нам потребуется сначала создать собственную форму документа, а затем воспользоваться возможностями встроенного языка.

Дело в том, что до сих пор мы использовали предопределенные формы объектов, которые система «1С:Предприятие» по умолчанию создавала для нас сама. Теперь же у нас возникла необходимость слегка изменить логику работы формы документа, поэтому нам нужно создать свою собственную форму документа ПриходнаяНакладная для того, чтобы в ней с помощью встроенного языка описать тот алгоритм, который нам нужен. И система будет использовать нашу форму вместо формы по умолчанию.

В режиме «Конфигуратор»

Форма документа

Вернемся в конфигуратор и откроем окно редактирования объекта конфигурации Документ ПриходнаяНакладная.

В этом окне нас интересует закладка Формы.

Как мы видим, ни одна из основных форм документа пока не задана. Для того чтобы создать форму документа, нажмем кнопку открытия  со значком лупы в поле ввода или кнопку Добавить над списком форм (рис. 4.17).

Система вызовет еще один полезный инструмент разработчика – *конструктор формы* (рис. 4.18).

Этот инструмент также построен по принципу «мастеров»: ввод данных в определенной последовательности и передвижение кнопками Далее и Назад.

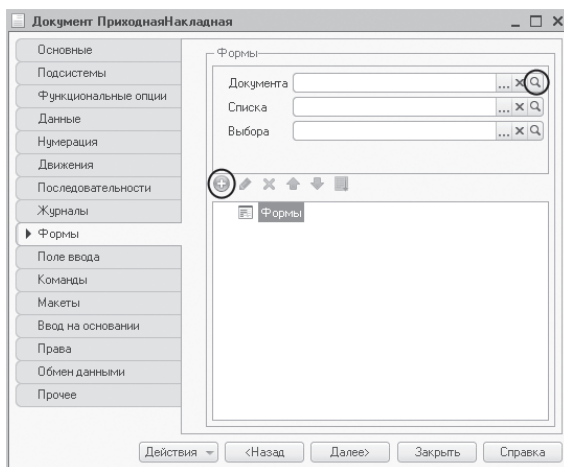


Рис. 4.17. Создание формы документа

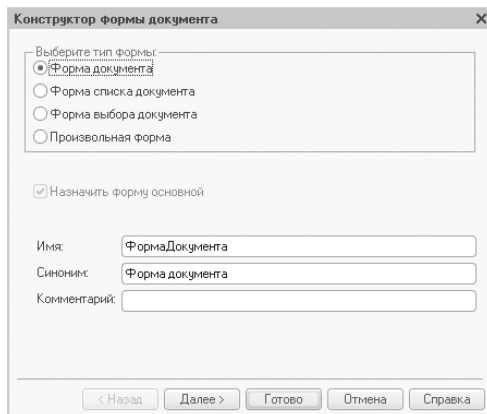


Рис. 4.18. Конструктор форм

Выберем тип формы Форма документа и нажмем кнопку Готово, согласившись тем самым со всем, что нам предложила система (см. рис. 4.18).

Обратите внимание, что в дереве объектов конфигурации у объекта конфигурации Документ ПриходнаяНакладная появилась форма ФормаДокумента (рис. 4.19), а на экране открылось окно редактора форм, содержащее эту форму (рис. 4.20).

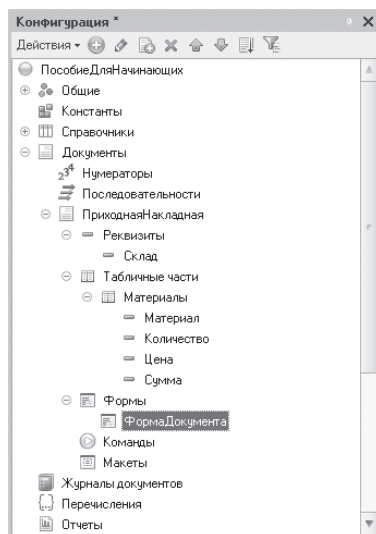


Рис. 4.19. Новая форма документа в конфигураторе

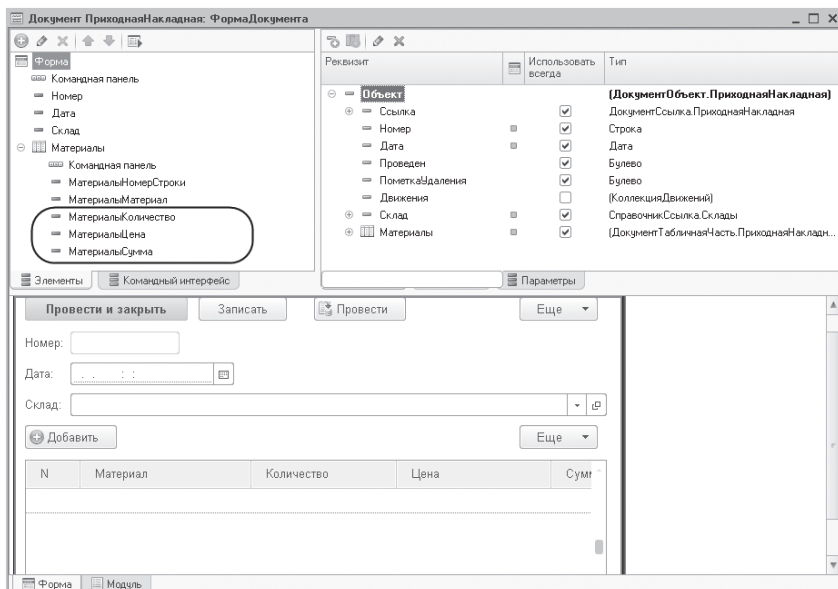


Рис. 4.20. Окно редактора форм

Редактор форм объединяет несколько окон взаимосвязанных между собой редакторов. Мы не будем здесь разбирать подробно работу с редактором форм, а коснемся только тех моментов, которые нужны для выполнения простейших действий, связанных с нашей задачей.

ПОДРОБНЕЕ

За более подробной информацией о работе с редактором форм следует обратиться к документации «1С:Предприятие 8.3.22 Руководство разработчика», раздел 31.1 «Редактор формы».

Пока мы рассмотрим только окно формы документа в режиме просмотра, расположенное внизу, и окно редактора элементов формы, расположенное слева в верхней части окна редактора форм.

Прежде всего, необходимо понимать, что при разработке форм объектов конфигурации разработчик не имеет возможности нарисовать форму. Он может только указать, из каких элементов будет состоять форма, а система уже сама самостоятельно расположит эти элементы в форме.

Элементы формы в верхнем левом окне редактора форм образуют иерархическую структуру, из которой следует, что чем выше в списке находится элемент, тем выше и левее на форме он будет располагаться.

Эта структура редактируется на закладке *Элементы* и позволяет управлять отображением и редактированием данных в форме.

Мы видим, что на основе описания в конфигурации документа *ПриходнаяНакладная* система создала структуру элементов, которая определяет, как будет выглядеть форма.

Эти элементы имеют разное назначение и разное поведение. Однако все они служат для того, чтобы отображать информацию, хранящуюся в базе данных, и организовывать интерактивную работу с этой информацией.

Вы можете попробовать перетащить мышью поля в дереве элементов и поменять местами, например, реквизиты табличной части. Результат изменений сразу отразится в форме документа в нижней части редактора форм. При этом разработчику не нужно задумываться над вопросами конкретного (до пикселя) расположения того или иного элемента формы, его размеров и привязки к другим элементам. Эту работу берет на себя система.

Но разработчик может через палитру свойств изменить свойства элемента, которые повлияют на его отображение в форме. Он может также изменить структуру элементов формы – создать новое поле, группу полей, добавить табличную часть, связав эти элементы с данными формы.

Но пока нам ничего этого не нужно делать. Нас интересуют три элемента табличной части: МатериалыКоличество, МатериалыЦена и МатериалыСумма (см. рис. 4.20).

Мы хотим, чтобы каждый раз, когда меняется значение в поле Количество или в поле Цена, в поле Сумма автоматически устанавливалось значение, равное произведению значений полей Количество и Цена.

Очевидно, что для этого нужно написать на встроенном языке команду, похожую на $\text{Сумма} = \text{Количество} * \text{Цена}$, которая будет выполняться при изменении значения поля Количество или Цена. Но как «поймать» эти моменты изменения?

Обработчик события

Как мы уже видели, система сама умеет работать с теми объектами, которые описаны в дереве конфигурации: показывать их данные, добавлять новые элементы и пр. То есть у нее есть некие «стандартные представления» о том, как это все должно работать.

Но, как правило, разработчиков эти «стандартные представления» устраивают только в самых простых случаях. Реальные задачи гораздо разнообразнее. Поэтому у системы существуют *события*, которые связаны с самыми различными моментами ее «стандартного» функционирования. В том числе события, связанные с функционированием форм и элементов, расположенных в этих формах.

Используя встроенный язык, разработчик может «вклиниться» в эти события и описать собственный алгоритм того, что должно происходить при наступлении этого события. Что мы сейчас и сделаем.

Дважды щелкнем на элементе формы МатериалыКоличество или правой кнопкой мыши откроем для него палитру свойств (пункт контекстного меню Свойства).

Прокрутив список до конца, мы увидим перечень событий, которые могут быть связаны с этим полем.

Очевидно, что нам нужно событие ПриИзменении, которое возникает после изменения значения поля. Найдем его в списке событий

и нажмем кнопку открытия  со значком лупы в поле ввода (рис. 4.21).

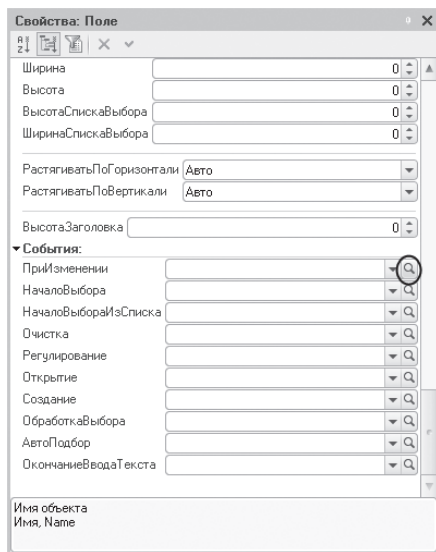


Рис. 4.21. Создание обработчика события «ПриИзменении» поля табличной части «Количество»

Система предложит окно для выбора типа обработчика события, который мы хотим создать (рис. 4.22).

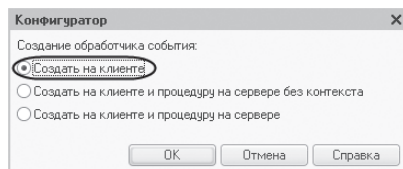


Рис. 4.22. Окно для выбора типа создаваемого обработчика события

Оставим без изменения вариант Создать на клиенте, предложенный по умолчанию. Пока мы не будем более подробно рассматривать этот момент. Вернемся к этому вопросу позднее в разделе «Директивы компиляции».

После этого система создаст шаблон процедуры *обработчика* этого события в модуле нашей формы и откроет закладку Модуль редактора формы (рис. 4.23).

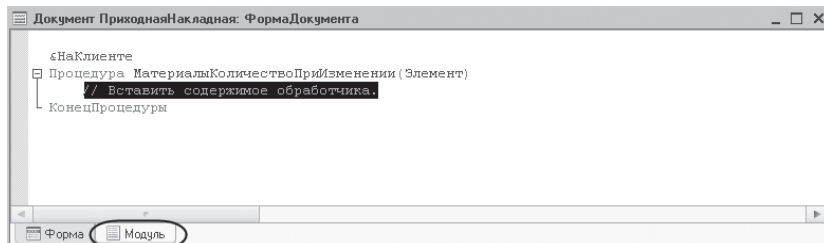


Рис. 4.23. Шаблон обработчика события «ПриИзменении» поля табличной части «Количество»

Модуль – это «хранилище» для текста программы на встроенном языке. Модулей существует несколько, и каждый из них предназначен для описания алгоритмов, относящихся к тому или иному моменту работы программы. В данном случае это модуль формы, так как обработчики всех интерактивных событий, связанных с элементами формы, помещаются именно в модуль формы.

В модуль формы, в процедуру `МатериалыКоличествоПриИзменении()` мы и добавим следующий текст (листинг 4.1).

Листинг 4.1. Процедура «МатериалыКоличествоПриИзменении()»

```
СтрокаТабличнойЧасти = Элементы.Материалы.ТекущиеДанные;  
СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;
```

Объясним назначение этих строк.

В первой строке мы сначала создаем переменную `СтрокаТабличнойЧасти`, в которую будет помещен объект, содержащий данные, находящиеся в строке табличной части, которую нам нужно пересчитать.

Мягкая типизация данных встроенного языка позволяет сделать это, не объявляя переменную и ее тип заранее. Мы создаем переменную прямо по ходу работы, и ее тип определяется типом значения, которое она содержит.

Поскольку мы находимся в модуле формы, то в нем доступны все свойства и методы объекта встроенного языка `УправляемаяФорма`. Поэтому мы можем обращаться к ним напрямую. В данном случае

после знака равенства мы обращаемся к коллекции элементов формы, используя одно из свойств объекта `УправляемаяФорма` – свойство `Элементы`.

Коллекция элементов формы является объектом встроенного языка `ВсеЭлементыФормы`, содержащим все элементы формы. То есть это программный аналог корня дерева элементов формы.

Каждый элемент формы можно получить, указав его имя в качестве свойства этого объекта, то есть через точку от него. В данном случае мы обращаемся к табличной части документа `Материалы` (`Элементы.Материалы`).

Табличная часть документа представляет собой объект встроенного языка `ТаблицаФормы`. Получить ту строку, в которой в настоящее время осуществляется редактирование, можно при помощи свойства программного объекта `ТаблицаФормы` – `ТекущиеДанные` (`Элементы.Материалы.ТекущиеДанные`).

Таким образом, в результате выполнения первой строки процедуры обработчика переменная `СтрокаТабличнойЧасти` будет содержать объект `ДанныеФормыСтруктура`. Этот объект содержит данные, находящиеся в текущей строке табличной части документа (`Элементы.Материалы.ТекущиеДанные`).

Получив этот объект, мы можем обратиться к данным конкретной колонки табличной части, указав имя колонки в качестве свойства объекта. Например, используя обращение `СтрокаТабличнойЧасти.Количество`, мы получаем число, которое находится в редактируемой строке в колонке `Количество`.

Таким образом, во второй строке процедуры обработчика вычисляется значение колонки `Сумма` как произведение значений колонок `Количество` и `Цена`.

В режиме «1С:Предприятие»

Теперь посмотрим, как это работает. Запустим «1С:Предприятие» в режиме отладки, откроем список документов `Приходные накладные` и откроем любой из двух созданных нами документов. Если теперь вы поменяете количество в любой строке документа, то сумма в строке будет пересчитана автоматически.

Одна процедура для обработки нескольких событий

Итак, мы убедились, что при изменении количества в любой строке документа Приходная накладная сумма в этой строке пересчитывается автоматически.

Замечательно. Но теперь хотелось бы и для поля Цена сделать то же самое. А если заглянуть вперед, то мы увидим, что подобное автоматическое заполнение поля Сумма может нам понадобиться и в других документах.

Поэтому лучше будет поместить расчет суммы в некоторое «общедоступное» место, чтобы разные документы, имеющие аналогичные реквизиты табличной части, могли использовать этот алгоритм.

Для описания таких «общедоступных» мест служат объекты конфигурации *Общий модуль*, расположенные в ветке *Общие > Общие модули*. Процедуры и функции, содержащиеся в этих модулях, могут быть доступны для любых объектов конфигурации.

Поэтому создадим общий модуль и перенесем в него нашу процедуру расчета суммы. А в документе просто оставим вызовы этой процедуры из общего модуля.

В режиме «Конфигуратор»

Общий модуль

Добавим объект конфигурации *Общий модуль*.

Для этого раскроем ветвь *Общие* в дереве объектов конфигурации, нажав на + слева от нее. Затем выделим ветвь *Общие модули* и нажмем кнопку *Добавить* в командной панели окна конфигурации (рис. 4.24).

Откроется окно для ввода текста модуля и окно палитры его свойств. Дадим имя модулю – *РаботаСДокументами* и установим в его свойствах флажок *Клиент* (управляемое приложение), а флажок *Сервер* снимем. Это означает, что экземпляры этого модуля будут скомпилированы в контексте тонкого клиента и в контексте веб-клиента (рис. 4.25).

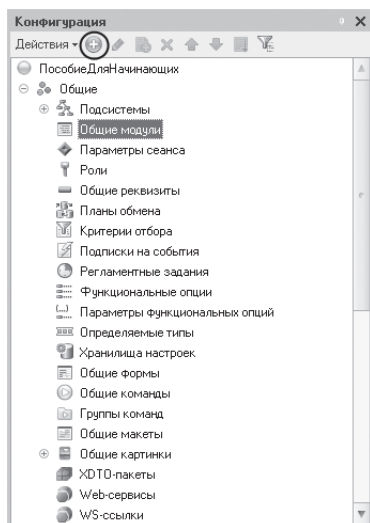


Рис. 4.24. Создание общего модуля в дереве объектов конфигурации

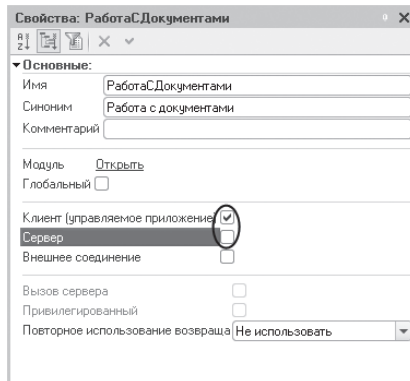


Рис. 4.25. Свойства общего модуля

Внесем в модуль следующий текст (листинг 4.2).

Листинг 4.2. Процедура «РассчитатьСумму()»

```

Процедура РассчитатьСумму(СтрокаТабличнойЧасти) Экспорт
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество
                                                * СтрокаТабличнойЧасти.Цена;
КонецПроцедуры
  
```

Прокомментируем этот код. В процедуру `РассчитатьСумму()` мы передаем переменную `СтрокаТабличнойЧасти`, которую мы определили в обработчике события `ПриИзменении` поля `Количество`. Она содержит данные редактируемой строки табличной части документа `ПриходнаяНакладная`.

Теперь, используя эту переменную, мы можем получить доступ к данным колонок табличной части и рассчитать сумму как произведение цены на количество.

Ключевое слово `Экспорт` в заголовке процедуры указывает на то, что эта процедура может быть доступна из других программных модулей.

Теперь в модуле нашей формы изменим текст обработчика `МатериалыКоличествоПриИзменении` (листинг 4.3).

Листинг 4.3. Процедура «`МатериалыКоличествоПриИзменении()`»

```
&НаКлиенте
Процедура МатериалыКоличествоПриИзменении(Элемент)
    СтрокаТабличнойЧасти = Элементы.Материалы.ТекущиеДанные;
    РаботаСДокументами.РассчитатьСумму(СтрокаТабличнойЧасти);
КонецПроцедуры
```

Мы видим, что первая строка процедуры осталась без изменений. А во второй строке вместо непосредственного расчета суммы мы вызываем процедуру `РассчитатьСумму()` из общего модуля `РаботаСДокументами` и передаем ей в качестве параметра текущую строку табличной части.

Проверим, как это работает, и убедимся, что ничего не изменилось.

Теперь осталось и для поля `Цена` установить такой же обработчик. Так как однажды мы уже написали в модуле формы нужную нам процедуру, то мы просто могли бы сопоставить ее также и другому событию другого элемента управления, расположенного в форме. Однако стандарты разработки конфигураций фирмы «1С» не допускают такого решения.

УЗНАЙ БОЛЬШЕ!

Согласно стандартам разработки фирмы «1С» у каждого события должен быть свой обработчик. Если одинаковые действия должны выполняться при изменении разных элементов управления (например, при нажатии нескольких кнопок), то в этом случае следует поступать следующим образом:

- создается отдельная процедура (функция), выполняющая необходимые действия;

- для каждого элемента управления создается отдельный обработчик с именем, назначаемым по умолчанию;
- из каждого обработчика вызывается требуемая процедура (функция).

Поэтому мы создадим обработчик события **ПриИзменении** для поля табличной части **МатериалыЦена** так же, как мы делали это для поля **МатериалыКоличество**, и повторим в нем вызов процедуры **РассчитатьСумму** из общего модуля (листинг 4.4).

Листинг 4.4. Процедура «МатериалыЦенаПриИзменении()»

```
&НаКлиенте  
Процедура МатериалыЦенаПриИзменении(Элемент)  
    СтрокаТабличнойЧасти = Элементы.Материалы.ТекущиеДанные;  
    РаботаСДокументами.РассчитатьСумму(СтрокаТабличнойЧасти);  
КонецПроцедуры
```

В режиме «1С:Предприятие»

Запустим «1С:Предприятие» в режиме отладки и убедимся, что теперь сумма в строках табличной части документов **ПриходнаяНакладная** пересчитывается как при изменении количества, так и при изменении цены.

Документ «Оказание услуги»

Теперь мы аналогичным образом создадим второй документ, необходимый нам, – **Оказание услуги**. Для этого потребуется выполнить уже знакомые нам действия, которые мы выполняли по созданию документа «**Приходная накладная**».

В режиме «Конфигуратор»

Добавим новый объект конфигурации **Документ** и назовем его **ОказаниеУслуги**.

На закладке **Основные** определим, как будет представлен документ в интерфейсе «1С:Предприятия».

Представление объекта задавать не будем, вместо него будет использоваться **Синоним** объекта.

Представление списка зададим как Оказание услуг. На закладке Подсистемы отметим, что документ будет доступен в подсистемах Оказание услуг и Бухгалтерия. На закладке Данные создадим реквизиты документа:

- Склад, тип СправочникСсылка.Склады. Выберем для свойства Значение заполнения предопределенный элемент Основной справочника Склады;
- Клиент, тип СправочникСсылка.Клиенты. Установим свойство Проверка заполнения в значение Выдавать ошибку;
- Мастер, тип СправочникСсылка.Сотрудники. Установим свойство Проверка заполнения в значение Выдавать ошибку.

Создадим табличную часть этого документа ПереченьНоменклатуры с реквизитами:

- Номенклатура, тип СправочникСсылка.Номенклатура;
- Количество, тип Число, длина 15, точность 3, неотрицательное;
- Цена, тип Число, длина 15, точность 2, неотрицательное;
- Сумма, тип Число, длина 15, точность 2, неотрицательное.

Установим для табличной части в целом и для каждого ее реквизита свойство Проверка заполнения в значение Выдавать ошибку.

На закладке Формы создадим основную форму документа.

Для поля ПереченьНоменклатурыКоличество создадим обработчик события ПриИзменении (выполняющийся на клиенте), в котором будем вызывать процедуру РассчитатьСумму из общего модуля РаботаСДокументами.

При этом откроется модуль формы с шаблоном обработчика события ПереченьНоменклатурыКоличествоПриИзменении, который мы пока заполнять не будем, а перейдем в окно элементов формы на закладку Форма и аналогичным образом создадим обработчик события ПереченьНоменклатурыЦенаПриИзменении для поля ПереченьНоменклатурыЦена.

Далее модуль формы документа ОказаниеУслуги нужно заполнить следующим образом (листинг 4.5).

Листинг 4.5. Модуль формы документа «ОказаниеУслуги»

&НаКлиенте

Процедура ПереченьНоменклатурыКоличествоПриИзменении(Элемент)

СтрокаТабличнойЧасти = Элементы.ПереченьНоменклатуры.ТекущиеДанные;

РаботаСДокументами.РассчитатьСумму(СтрокаТабличнойЧасти);

КонецПроцедуры

&НаКлиенте

Процедура ПереченьНоменклатурыЦенаПриИзменении(Элемент)

СтрокаТабличнойЧасти = Элементы.ПереченьНоменклатуры.ТекущиеДанные;

РаботаСДокументами.РассчитатьСумму(СтрокаТабличнойЧасти);

КонецПроцедуры

В заключение отредактируем командный интерфейс, чтобы в разделе Оказание услуг была доступна команда создания новых документов.

Для этого откроем окно редактирования объекта конфигурации Подсистема ОказаниеУслуг и нажмем кнопку Командный интерфейс. В открывшемся окне отразятся все команды данной подсистемы.

В группе Панель действий.Создать включим видимость у команды Оказание услуги: создать (рис. 4.26).

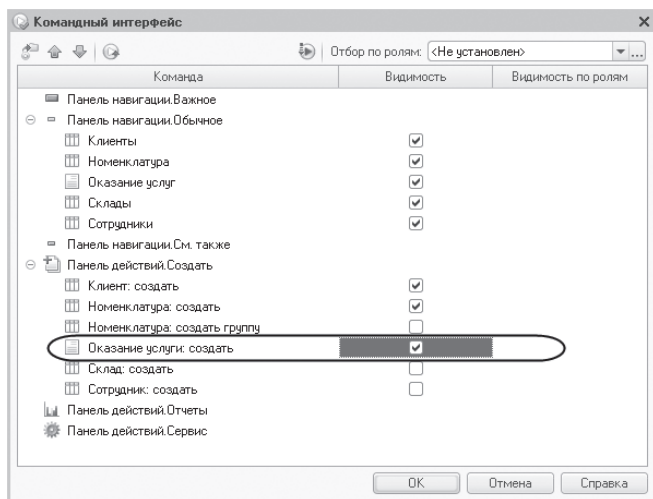


Рис. 4.26. Настройка командного интерфейса подсистемы

В результате наших действий в дереве объектов конфигурации документ ОказаниеУслуги будет выглядеть следующим образом (рис. 4.27).

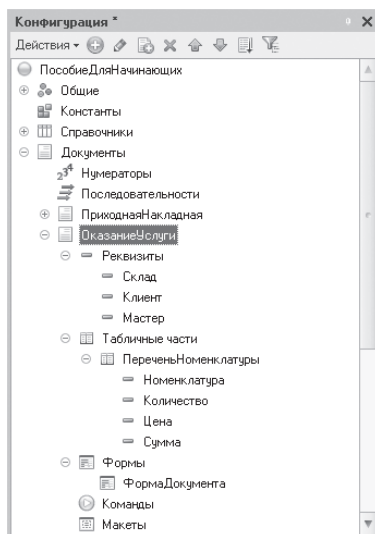


Рис. 4.27. Документ «ОказаниеУслуги» в дереве объектов конфигурации

В режиме «1С:Предприятие»

Запустим «1С:Предприятие» в режиме отладки.

В разделе Оказание услуг выполним команду создания документа Оказание услуги из подменю Создать и заполним его следующим образом (рис. 4.28).

The screenshot shows the 'Оказание услуги (создание)' form. It includes the following fields and controls:

- Buttons: Провести и закрыть, Записать, Провести, Еще -
- Number: []
- Date: 12.10.2022 0:00:00
- Warehouse: Основной
- Client: Иванов Михаил Юрьевич
- Master: Деловой Иван Сергеевич
- Buttons: Добавить, [Up], [Down]
- Search: Поиск (Ctrl+F), [X], Еще -
- Table:

N	Номенклатура	Количество	Цена	Сумма
1	Транзистор Philips ZN2369	1,000	3,00	3,00

Рис. 4.28. Создание документа «Оказание услуги № 1»

Обратите внимание, что склад Основной подставляется по умолчанию, а для полей Мастер и Клиент выполняется проверка заполнения. А также при вводе цены и количества в табличную часть документа Оказание услуги сумма пересчитывается по нашему алгоритму.

Нажмем Провести и закрыть.

Контрольные вопросы

- ☒ Для чего предназначен объект конфигурации «Документ»?
- ☒ Какими характерными особенностями обладает документ?
- ☒ Для чего предназначены реквизиты и табличные части документа?
- ☒ Какие существуют основные формы документа?
- ☒ Что такое проведение документа?
- ☒ Как создать объект конфигурации «Документ» и описать его основную структуру?
- ☒ Как создать новый документ и заполнить его данными?
- ☒ Как создать собственную форму документа?
- ☒ Что такое конструктор форм?
- ☒ Что такое редактор форм?
- ☒ Что такое элементы формы?
- ☒ Что такое события и с чем они связаны?
- ☒ Что такое обработчик события и как его создать?
- ☒ Что такое модуль и для чего он нужен?
- ☒ Зачем нужны общие модули?
- ☒ Что такое типобразующие объекты?